



Instructor Name

Zaheredin S. Mussa

Qualification

BSc. & MSc. in Computer Science

Email

Zaheredin.Shifa@amu.edu.et

Address

*Arbaminch University Institute of
Technology*



OBJECT ORIENTED PROGRAMMING

CHAPTER ONE

Introduction to
Object-Oriented Programming

CHAPTER TWO

Basics in Java Programming

CHAPTER THREE

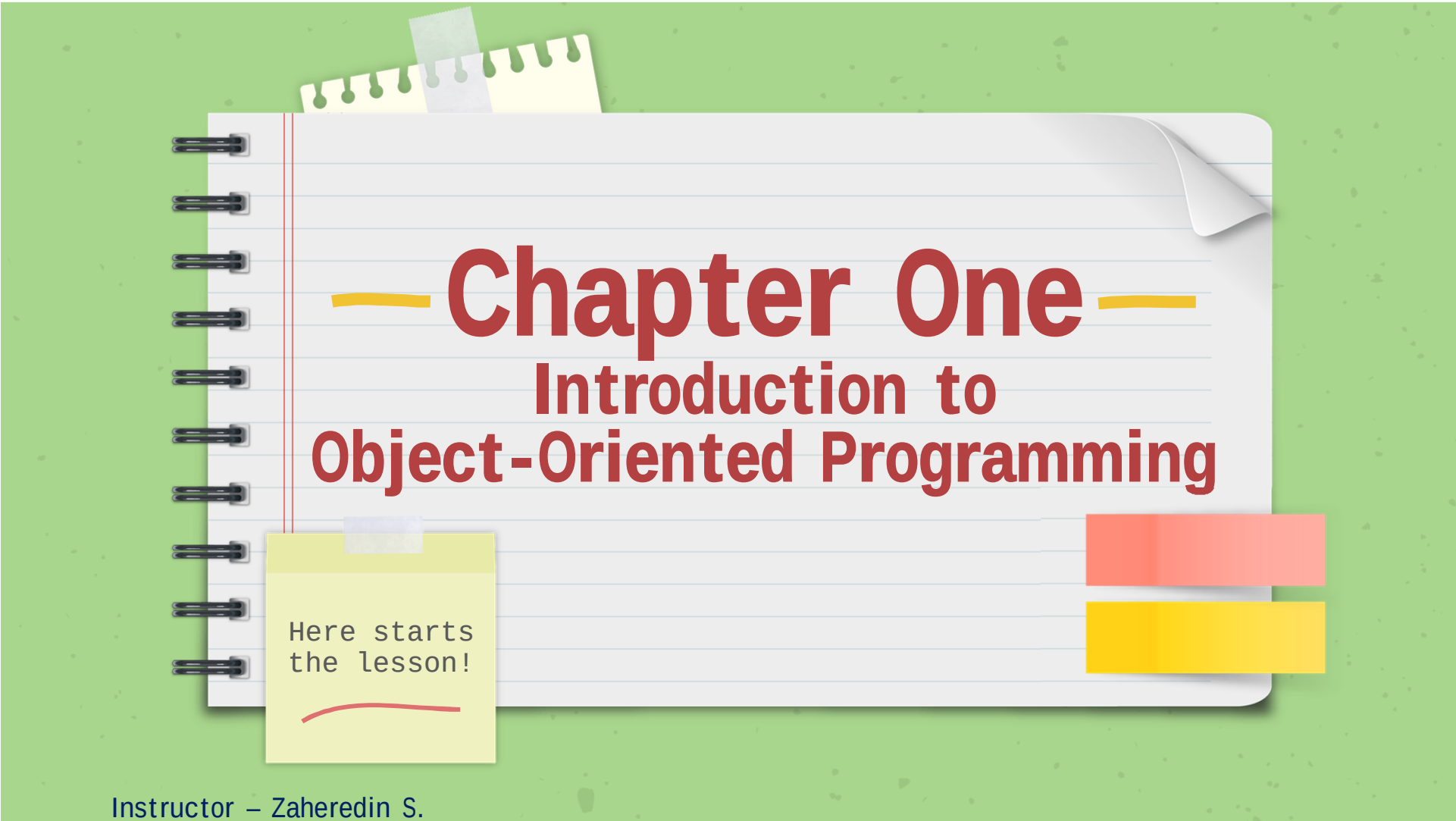
Decision and Repetition
Statements

CHAPTER FOUR

Objects and Classes

CHAPTER FIVE

OOP Concepts



Chapter One

Introduction to Object-Oriented Programming

Here starts
the lesson!

Instructor – Zaheredin S.

Chapter One Contents!



Types of programming paradigm

Structural Vs Object Oriented Programming Paradigms

Chapter One Contents!



Overview of Java Programming

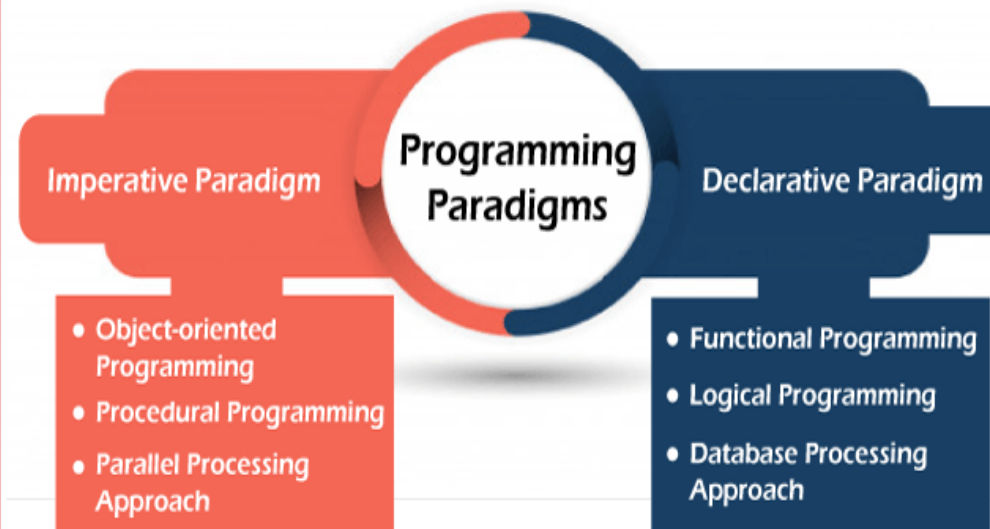
Overview of OO principles

Types of programming paradigm

What is a "programming paradigm"?

- It is a set of principles, concepts, and techniques that guide the development of software systems.
- A Programming paradigm is all about different approaches used to find a software solution to the problem or also we can say,
- It is a method to solve a problem using tools and techniques that are available to us following some approach.
- It is all about the writing style and organizing the program code in a specific way.

Types of Programming Paradigms



Cont...

Imperative Programming

- Imperative programming is a **programming paradigm** where the programmer tells the computer how to perform a task **step-by-step** using **specific instructions** or **commands**.
- **Examples** of this paradigm, **Java**, **C**, **C++**, **Python**, etc.

Declarative Programming

- It is a programming paradigm where programmer specifies what the program should accomplish without necessarily specifying how to accomplish it.
- **Examples** of this paradigm, **Prolog**, **Haskell**, **SQL**, etc.

Structural Vs OO Programming Paradigms

- **OOP** and **Structural Programming** are two popular programming paradigms that differ in their approach to software design and implementation.

A. Structured Programming

- It is a programming paradigm that focuses on dividing a program into smaller, more manageable pieces called functions or procedures.
- These functions have statements embraced inside curly braces.

Cont...

- Each of these **functions performs** a **subtask**. Usually, as each function represents a **specific functionality**, it is easier for the **programmer** to **test** and **debug** the code, but this can differ depending on the program.
- In the **structured programming**, the user can create his own **user-defined functions**.
- The **main function** calls the other functions. When there is a **function call**, the **control** is passed to **that function**. After completing the function, the **control** passes back to the **main program**.

Cont...

- In structural programming, the program is seen as a sequence of instructions that are executed one after the other.
- The main features of structural programming include modularity, top-down design, and stepwise refinement.
- C Programming language and Pascal are two Programming Language example.

Cont...

```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {4, 2, 5, 1, 3};  
        bubbleSort(numbers);  
        for (int i = 0; i < numbers.length; i++) {  
            System.out.println(numbers[i]);  
        }  
    }  
}
```

Cont...

```
public static void bubbleSort(int[] arr) {  
    int n = arr.length;  
    for (int i = 0; i < n-1; i++) {  
        for (int j = 0; j < n-i-1; j++) {  
            if (arr[j] > arr[j+1]) {  
                int temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;  
            }  
        }  
    }  
}
```

Cont...

A. Object-oriented Programming (OOP)

- Object-Oriented Programming is a programming paradigm that focuses on modeling real-world entities as objects and defining their properties and behavior through the use of classes and methods.
- An object contains data in the form of fields (attributes) and the procedures (methods).
- The main features of OOP include inheritance, encapsulation, and polymorphism to make code more reusable, modular, and maintainable.

Cont...

- The object-oriented paradigm provides key benefits of **reusable code** and **code extensibility**.
- OOP can be achieved by using programming languages such as **Java**, **C++**, **C#**, **PHP**, and **Python**, etc.

Cont...

```
public class Car {  
    private String make;  
    private String model;  
    private int year;  
    public Car(String make, String model, int year) {  
        this.make = make; this.model = model; this.year = year;  
    }  
    public String getMake() {  
        return make;  
    }  
    public String getModel() {  
        return model;  
    }  
}
```

Cont...

```
public int getYear() {  
    return year;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car myCar = new Car("Honda", "Civic", 2021);  
        System.out.println("Make: " + myCar.getMake());  
        System.out.println("Model: " + myCar.getModel());  
        System.out.println("Year: " + myCar.getYear());  
    }  
}
```

Cont...

Difference Between Structured and Object Oriented Programming

Structured	Object Oriented
Focuses on breaking down a program into smaller functions that perform specific tasks	focuses on representing a program using a set of objects which encapsulates data and behaviour.
More suitable for smaller projects or programs	Easier to work collaboratively on large projects or programs

Cont...

Structured	Object Oriented
It provides less flexibility and abstraction	It provides more flexibility and abstraction
It is difficult to modify the structured programs	It is easier to modify the Object Oriented programs
Data is not secure in structured programming	But data is secure in object oriented programming
Difficult to reuse code	Easier to reuse code

Cont...

The four fundamental of OOP concepts

I. Data abstraction

- It refers to the process of **hiding complex implementation details** behind a **simpler interface**, allowing users to interact with the system at a **higher level of abstraction**. This can make the code easier to **understand** and **maintain**.

II. Encapsulation

- It is the process of **hiding** the **internal details** of an object and exposing only the **necessary information** through its interface.

Cont...

- This helps to keep the code **modular** and **maintainable**.

III. Inheritance

- It allows a **class** to **inherit** **properties** and **behavior** from **another class**, creating a **hierarchical relationship** between the classes. This can help to reduce **code duplication** and make the code **more modular**.

IV. Polymorphism

- It allows objects of **different classes** to be treated as if they were the **same type**, as long as they implement the **same methods** or **interface**. This can make the code more **flexible** and **extensible**.

1.3 Overview of Java Programming

What is Java?

- Java is an **object-oriented programming** language that was first released in **1995** by **Sun Microsystems**.
- Java is designed to be **platform-independent**, meaning that **Java code** can run on **any platform** that has a **Java Virtual Machine (JVM)** installed.
- Java is often used for developing **enterprise applications**, **web applications**, and **Android mobile applications**.

Cont...

- Java programs are typically **compiled** into **bytecode**, which is then executed by the **JVM** at **runtime**. This allows **Java code** to be **portable** and **easily distributed** across **different platforms**.

Java Example

```
class Simple{  
    public static void main(String args[]) {  
        System.out.println("Hello Java");  
    }  
}
```

Cont...

For executing any java program, you need to

- Download **JDK** and **install** it.
- **Set path** of the **jdk/bin** directory.
- create the java program as shown below.

```
class Simple{  
    public static void main(String[] args){  
        System.out.println("Hello Java");  
    }  
}
```

Output:
Hello Java

Save as **Simple.java**

To compile - **javac Simple.java**

To execute - **java Simple**

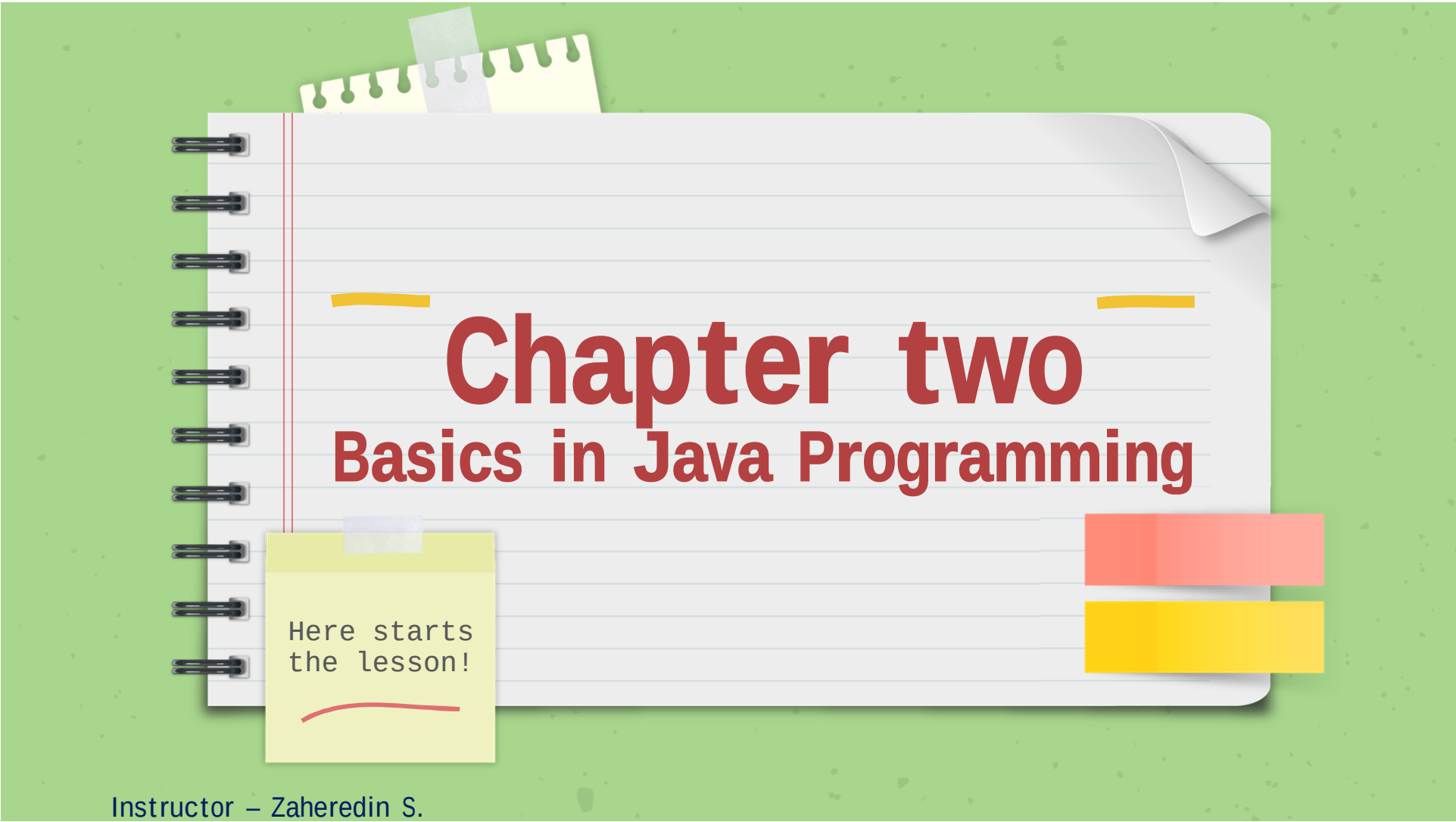
Understanding first java program

- class keyword is used to **declare a class** in java.
- public keyword is an **access modifier** which represents **visibility**
- static is a **keyword**. The core advantage of static method is that there is **no need to create object to invoke the static method**. Therefore, **it saves memory**.
- void is the **return type** of the method, it means it **doesn't return any value**.
- main represents **startup of the program**.
- String[] args is used to pass **command-line-arguments** to Java program
- System.out.println() is used to **print** statement.



END

Thank You!



Chapter two

Basics in Java Programming

Here starts
the lesson!

Instructor – Zaheredin S.

Chapter Two Contents!



Identifier, Keyword, Variable.

Chapter Two Contents!



Data types and constants

Identifiers, Keywords, Variables and Data types

A. Introduction to Java Identifiers

- All Java components require **names**. In Java, identifiers are the names that are used for the **identification of various entities** in java like **classes, methods, variables, packages**, etc.
- So in Java, Identifiers are the **class names, method names, variable names, package names, constant names**, etc.

Cont...

Rules for defining an Identifier in Java

- Identifiers can only contain alphanumeric characters [a-z] [A-Z] [0-9], dollar sign (\$) and underscore (_). No other character is allowed.
- Identifiers cannot start with a numeric value [0-9].
- The starting character should be alphabet [A-Z] [a-z], dollar (\$) or underscore (_).
- Identifiers should not contain spaces in their name.
- Identifiers are case sensitive. E.g. 'Result' and 'result' are considered two different identifiers.

Cont...

- A **key word** cannot be used as an identifier.

Examples of legal identifiers:

- age
- \$salary
- _value
- __1_value

Examples of **illegal** identifiers:

- 123abc
- -salary

Cont...

B. Java Reserved Keywords

- Java reserved keywords are **predefined words** that are used by java syntax for a **particular functionality**.
- In Java, **every reserved word** has a unique meaning and **functionality**.

The following list shows the reserved words in Java.

- These reserved words **may not** be **used as constant, variable**, or any other identifier names.

Java Keywords

Abstract	Assert	Boolean	Extends	Final	Finally
Byte	Case	Catch	For	Goto	If
Class	Const	Continue	Import	Instanceof	Int
Do	Double	Else	Long	Native	New
char	interface	void	Private	Protected	Public
default	package	implements	Short	Static	Strictfp
enum	return	throw	Switch	Synchronized	This
float	super	Try	Throws	Transient	
Volatile	While	break			

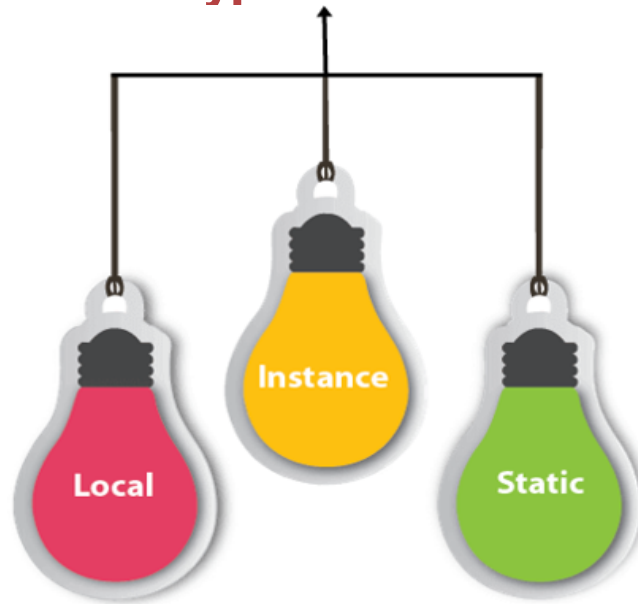
Cont...

C. What is "Variables"?

- Variables are **locations in memory** in which **values can be stored**.
- They have a **name**, a **type**, and a **value**.
- **Before** you can **use** a **variable**, you have to **declare it**.
- After **it is declared**, you can then **assign values** to it.
- The **value stored in a variable** can be **changed during execution**.

Java actually has **three kinds of variables:**

Three types of Variables



Cont...

I. Local Variable

- They are declared inside method definitions and destroyed after exiting from the block or when the call returns from the function.
- You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.
- Access modifiers cannot be used for local variables.
- Initialization of local variable is mandatory.

Cont...

Example of local variable

```
public class LocalVar {  
    public String myVar="Not local var";  
    public void myMethod(){  
        String myVar = "Inside Method";  
        System.out.println(myVar);  
    }  
    public static void main(String args[]){  
        LocalVar obj = new LocalVar();  
        obj.myMethod();  
        System.out.println(obj.myVar);  
    }  
}
```

OUTPUT

Inside Method
Not Local var

Cont...

II. Instance Variable (Non-Static field)

- They are **non-static variables** & are **declared inside a class** but **outside any methods, constructors or block**.
- These variables are **created when an object is created** with the use of the keyword 'new' and **destroyed when the object is destroyed**.
- Each instance (**objects**) of class has its own **copy of instance variable**.
- **Initialization** of instance variable is **not mandatory**.
- **Access modifiers can be given** for instance variables.

Cont...

Example of instance variable

```
public class InstanceVar{  
    String myVar="This is instance variable";  
    public static void main(String args[]){  
        InstanceVar obj1 = new InstanceVar();  
        InstanceVar obj2 = new InstanceVar();  
        System.out.println(obj1.myVar);  
        System.out.println(obj2.myVar);  
        obj2.myVar = "Changed Text";  
        System.out.println(obj1.myVar);  
        System.out.println(obj2.myVar);  
    }  
}
```

OUTPUT

```
This is instance variable  
This is instance variable  
This is instance variable  
Changed Text
```

Cont...

III. Class Variable (Static field)

- Static variables are declared using the static keyword within a class but outside any method constructors or block.
- The values of a class variable is apply to all that class's instances (and to the class itself) rather than having different values for each object.
- Static variables are created at the start of program execution and destroyed automatically when execution end.

Cont...

- Initialization of static variable is not mandatory.
- It can be accessed directly using class name.
- When declaring class variables as public static final, then variables names (constants) are all in upper case.
 - ❖ `public static final String DEPARTMENT = "Development";`
- If the static variables are not public and final, the naming syntax is the same as instance and local variables.
 - ❖ `private static double salary;`

Cont...

Example of class (static) variable

```
public class StaticVar{  
    public static String myVar="This is static variable";  
    public static void main(String args[]){  
        StaticVar obj1 = new StaticVar();  
        StaticVar obj2 = new StaticVar();  
        System.out.println(obj1.myVar);  
        System.out.println(obj2.myVar);  
        obj2.myVar = "Changed Text";  
        System.out.println(obj1.myVar);  
        System.out.println(obj2.myVar);  
    }  
}
```

OUTPUT

This is static variable
This is static variable
Changed Text
Changed Text

Example to understand the types of variables in java

```
class Simple {  
    static int a=20;  // static variable  
    int b=120;  // instance variable  
    public static void main(String args[]){  
        int c=100;  // local variable  
    }  
}
```

Cont...

Declaring Variable

- To use any **variable** in a Java program, you must **first declare it**.
- Variable declarations consist of a **type** and a **variable name**:
 - ❖ **int myAge;**
 - ❖ **String myName;**
 - ❖ **boolean isTired;**
- You can **string together variable** names with the same type. Example - **int x, y, z; String firstName, LastName;**

Cont...

- You can also give **each variable** an **initial value** when you declare it:

- ❖ `int myAge, mySize, numShoes = 28;`

- ❖ `String myName = "Laura";`

- ❖ `boolean isTired = true;`

- ❖ `int a = 4, b = 5, c = 6;`

Cont...

Rules for naming variables in Java

- Variable names in Java can **start with** a **letter**, an **underscore** (**_**), or a **dollar sign** (**\$**).
- They **cannot start with a number**.
- After the first character, your variable names can include any **letter** or **number**.
- Java language is **case-sensitive**. Hence, **age** and **AGE** are two different variables.

```
int age = 10;
```

```
int AGE = 15;
```

Cont...

- By convention, Java variables have **meaningful names**, often made up of **several words combined**.
- The first word is **lowercase**, but all following words have an **initial uppercase letter**:
- For Example -

```
long reallyBigNumber;
```

```
boolean currentWeatherStateOfPlanetXShortVersion;
```

Cont...

Assigning Values to Variables

- Once a variable has been declared, you can **assign a value** to that variable by using the **assignment operator** `=`.

```
int size = 14;
```

```
boolean tooMuchCaffiene = true;
```

Data Types and Constants in Java

A. Data Types in java

Data types in java can be divided into two parts.

1. Primitive data types

- ❖ Eight basic **primitive data types** such as, **integer**, **character**, **Boolean**, **float** etc.

2. Non-Primitive data types

- ❖ Which include **name of classes**, **arrays** and **interface**.

Cont...

Primitive data types

- The **eight primitive data types** handle common types for **byte**, **short**, **int**, **long**, **float**, **char**, **double**, and **Boolean** (true or false).
- Note that **these data types** are **machine-independent**, which means that you can **rely on their sizes** and characteristics to be **consistent** across your **Java programs**.

Cont...

- There are four Java integer types, each with different ranges of values (as listed in Table below).

Table 2.1 Integer types.

Type	Size	Range
Byte	8 bits	-128 to 127
Short	16 bits	-32,768 to 32,767
Int	32 bits	-2,147,483,648 to 2,147,483,647
Long	64 bits	-9223372036854775808 to 9223372036854775807

Cont...

- The **char** type is used for individual characters.
- Finally, the **Boolean** type can have one of two values, **true** or **false**.

Non-Primitive data types

- In addition to the **eight basic data types**, variables in Java can also be **declared to hold an instance** of a **particular class**:

for example :- String lastName; Font basicFont;
 OvalShape myOval;

Cont...

- Each of these variables can then hold only instances of the given class.
- As you create new classes, you can declare variables to hold instances of those classes (and their subclasses) as well.

Cont...

Table 2.2 Java Data types.

Data Type	Size	Data Type	Size
Byte	1 byte	Float	4 byte
Short	2 byte	Double	8 byte
Int	4 byte	Boolean	1 byte
Long	8 byte	char	2 byte

Cont...

B. Constants in Java

- A constant is an **identifier** that is similar to a variable except that it **holds the same value** during **its entire existence**.
- As the name implies, **it is constant, not variable**.
- The **compiler will issue an error** if you try to **change the value of a constant**.
- In Java, we use the **final modifier** to declare a **constant**

```
final int MIN_HEIGHT = 69;
```

Comments in Java

- Java supports **single-line** and **multi-line** comments very similar to c and c++.
- Java compiler **ignores** all characters available inside any comment.

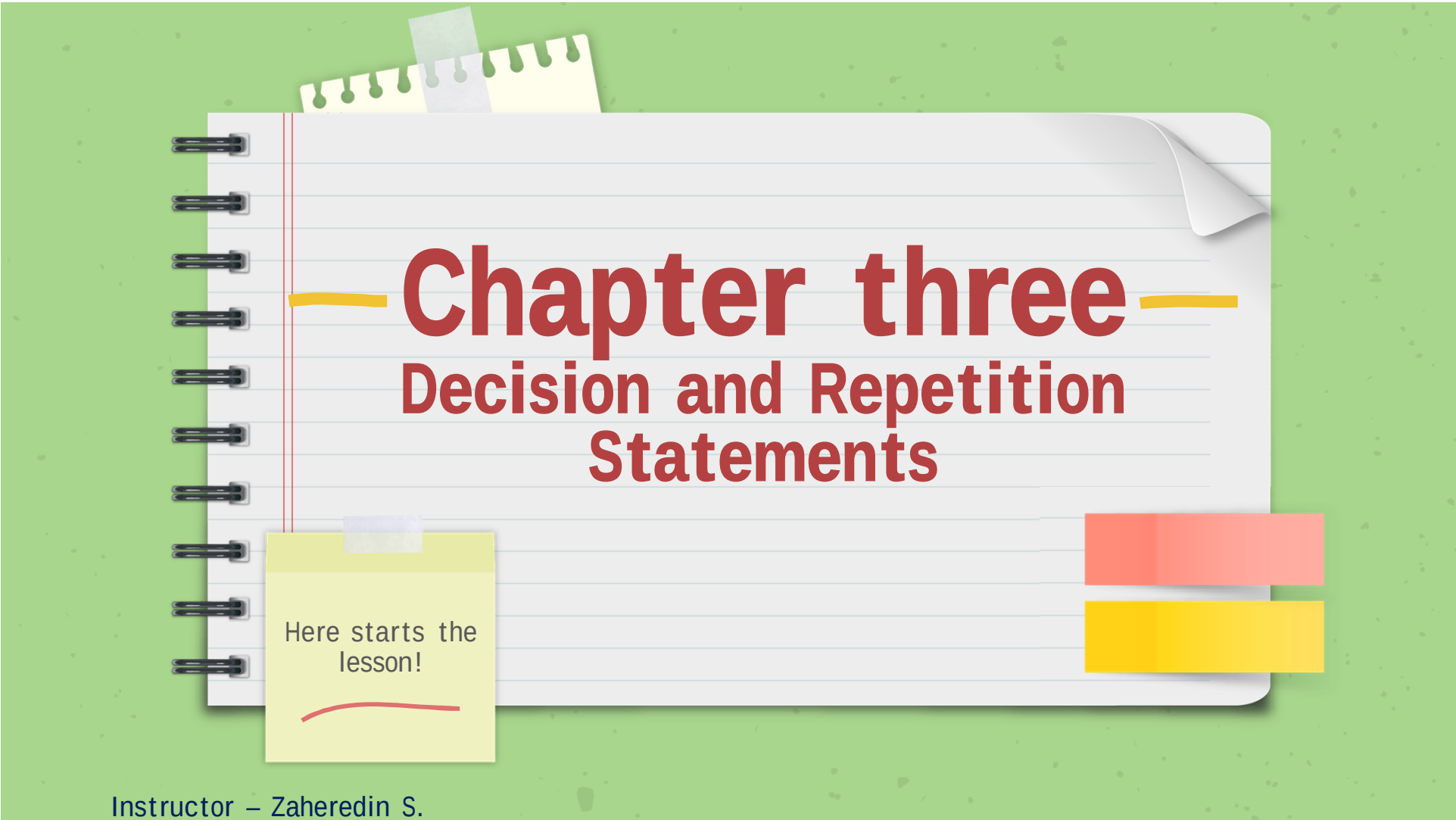
```
public class JavaProgram {  
    /* This is my first java  
    program.*/  
}
```

```
public      static      void  
main(String []args){  
    // Eg. of single line comment  
    System.out.println("Hello  
    World");  
}  
}
```



END

Thank You!



Chapter three

Decision and Repetition Statements

Here starts the
lesson!

Instructor – Zaheredin S.

Chapter Three Contents!



Overview of Java statements

Java Decision Making (if, switch)

Chapter Three Contents!



**Java Repetition Statements
(for, while & do while loop)**

Breaking control statement

Overview of Java statements

What is "Statements in java"?

- In Java, a statement is an executable instruction that tells the compiler what to do.
- All the following are simple Java statements:

```
int i = 1;
```

```
System.out.println ("This is motorcycle");
```

```
m.engineState = true;
```

- The most important thing to remember about Java statements is that each one ends with a semicolon.

Cont...

- If you forget the **semicolon** then your Java program **will not compile**.
- Java also has **compound statements**, or **blocks**, which can be placed wherever a single statement can.
- **Braces** surround **Block statements** (**{}**).

3.2 - Java Decision Making

- Java **decision-making statements** allow you to make a decision, based upon the result of a condition.
- There are **two types of decision-making statements** in Java.
- They are:
 - ❖ **if** statements
 - ❖ **switch** statements

if Statement

- It consists of a **Boolean expression** followed by **one or more statements**.
- The syntax of an if statement is:

```
if(Boolean_expression) {  
    //true Statements  
}  
//Statements
```

- If the **Boolean expression** evaluates to **true**, then the **block of code** inside the if statement **will be executed**.
- If **not**, the **first set of code** after the **end** of the if statement (after the closing curly brace) **will be executed**.

Example

```
public class Test {  
    public static void main(String args[]){  
        int x = 10;  
        if(x < 20){  
            System.out.print("This is if statement");  
        }  
    }  
}
```

Output
This is if statement

if else Statement

- **if** statement can be followed by an optional **else** statement, which executes when the Boolean expression is false.
- The syntax of if else statement is:

```
if(Boolean_expression){  
    //true Statements  
}  
else{  
    //Statements  
}
```

Example

```
public class Test {  
    public static void main(String args[]){  
        int x = 30;  
        if(x < 20){  
            System.out.print("This is if statement");  
        }  
        else{  
            System.out.print("This is else statement");  
        }  
    }  
}
```

Output

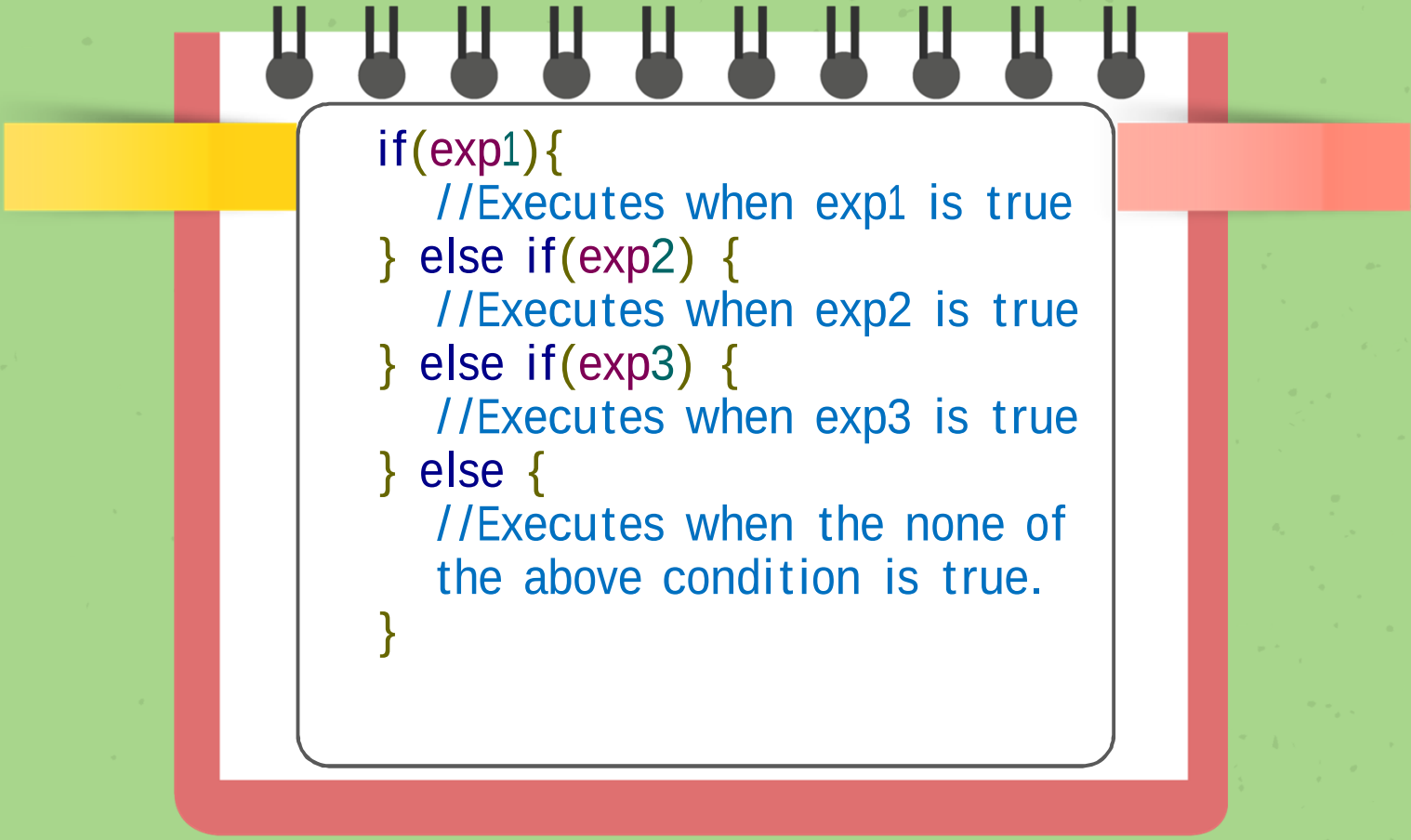
This is else statement

The if...else if...else Statement

- An **if statement** can be followed by an optional **else if...else statement**, which is very useful to test various conditions using **single if...else if statement**.

When using if, else if, else statements there are few points to keep in mind.

- An if can have **zero or one else's** and it must **come after any else if's**.
- An if can have zero to many **else if's** and they must **come before the else**.
- Once an **else if succeeds**, none of the **remaining else if's** or **else's** will be **tested**.
- Syntax of an if...else is:



```
if(exp1){  
    //Executes when exp1 is true  
} else if(exp2) {  
    //Executes when exp2 is true  
} else if(exp3) {  
    //Executes when exp3 is true  
} else {  
    //Executes when the none of  
    the above condition is true.  
}
```

Example

```
public class Test {  
    public static void main(String args[]){  
        int x = 30;  
        if(x == 10){  
            System.out.print("Value of X is 10");  
        }else if(x == 20){  
            System.out.print("Value of X is 20");  
        }else if(x == 30){  
            System.out.print("Value of X is 30");  
        }else{  
            System.out.print("This is else statement");  
        }  
    }  
}
```

Output
Value of X is 30

Nested if...else Statement

- It is always legal to **nest if-else statements** which means you can use **one if or else if** statement **inside another if or else if** statement.
- **The syntax for a nested if...else is as follows:**

```
if(exp-1){  
    //Executes when exp-1 is true  
    if(exp-2){  
        //Executes when exp-2 is true  
    }  
}
```

- You can nest **else if...else** in the similar way as we have **nested if** statement.

Example

```
public class Test {  
    public static void main(String args[]){  
        int x = 30;  
        int y = 10;  
        if(x == 30){  
            if(y == 10){  
                System.out.print("X = 30 and Y = 10");  
            }  
        }  
    }  
}
```

Output

X = 30 and Y = 10

The switch Statement

- A switch statement allows a variable to be tested for equality against a list of values.
- Each value is called a case, and the variable being switched on is checked for each case.
- The syntax of enhanced for loop is:

```
switch(expression){  
  case value : //Statements  
    break;  
  case value : //Statements  
    break;  
    //You can have any number  
    of case statements.  
  default : //Optional  
    //Statements  
}
```

Cont...

The following rules apply to a switch statement:

- The variable used in a switch statement **can only** be a **byte, short, int, or char**.
- You can have **any number of case statements** within a switch. Each case is **followed by the value** to be compared to and a colon.
- The value for a case must be the **same data type** as the variable in the switch and it must be a **constant** or a **literal**.

Cont...

- When the variable being switched on is equal to a case, the statements following that case will execute until a break statement is reached.
- When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
- Not every case needs to contain a break. If no break appears, the flow of control will fall through to subsequent cases until a break is reached.

Cont...

- A switch statement can have an optional default case, which must appear at the end of the switch.
- The default case can be used for performing a task when none of the cases is true.
- No break is needed in the default case.

Example

```
public class Test {  
    public static void main(String args[]){  
        char grade = 'C';  
        switch(grade) {  
            case 'A' : System.out.println("Excellent!"); break;  
            case 'B' : System.out.println("Very Good"); break;  
            case 'C' : System.out.println("Well done"); break;  
            default : System.out.println("Invalid grade");  
        }  
        System.out.println("Your grade is " + grade);  
    }  
}
```

Output
Well done
Your grade is C

3.3 - Java Repetition Statements

- There may be a situation when we need to **execute** a **block of code several number of times**, and is often referred to as a **loop**.
- Java has very flexible **three looping mechanisms**.
 - ❖ **while** Loop
 - ❖ **do...while** Loop
 - ❖ **for** Loop

The while Loop

- A while loop is a **control structure** that allows you to **repeat a task a certain number of times**.
- The syntax of a while loop is:

```
while(Boolean_expression)
{
    //Statements
}
```

- When executing, **if** the **boolean_expression** result is **true**, then the actions **inside the loop will be executed**. This will continue as long as the expression result is true.
- **When** the **expression** result is **false**, the **loop body** will be **skipped** and the first **statement after the while loop will be executed**.

Example

```
public class Test {  
    public static void main(String args[]) {  
        int x = 10;  
        while( x < 19 ){  
            System.out.print("value of x : " + x );  
            x++;  
            System.out.print("\n");  
        }  
    }  
}
```

Output

```
value of x : 10  
value of x : 11  
value of x : 12  
value of x : 13  
value of x : 14  
value of x : 15  
value of x : 16  
value of x : 17  
value of x : 18
```

The do...while Loop

- A **do...while** loop is similar to a while loop, except that a **do...while** loop is **guaranteed to execute at least one time**.
- The syntax of a do while loop is:

```
do{  
    //Statements  
}  
while(expression);
```
- The **expression** appears at the **end of the loop**, so the **statements** in the loop **execute once before** the **expression is tested**.
- If the **expression is true**, the flow of control **jumps back up to do**, and the **statements** in the loop **execute again**. This process repeats until the **expression is false**.

Example

```
public class Test {  
    public static void main(String args[]){  
        int x = 10;  
        do{  
            System.out.print("value of x : " + x );  
            x++;  
            System.out.print("\n");  
        }  
        while( x < 19 );  
    }  
}
```

Output

value of x : 10
value of x : 11
value of x : 12
value of x : 13
value of x : 14
value of x : 15
value of x : 16
value of x : 17
value of x : 18

Cont...

The for Loop

- A for loop is a **repetition control structure** that allows you to efficiently write a loop that needs to **execute a specific number of times**.
- A for loop is useful when you know **how many times** a **task** is to be **repeated**.
- The syntax of for loop is:

```
for(initialization; Boolean_expression; update){  
    //Statements  
}
```

Cont...

- The **initialization step is executed first**, and only once. This step allows you to declare and initialize any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
- **Next, the Boolean expression is evaluated.** If it is **true**, the **body of the loop is executed**. If it is **false**, the body of the loop **does not execute** and flow of control **jumps** to the **next statement** past the for loop.

Cont...

- After the body of the for loop executes, the flow of control jumps back up to update the statement. This statement allows you to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the Boolean expression.
- The Boolean expression is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then update step, then Boolean expression). After the Boolean expression is false, the for loop terminates.

Example

```
public class Test {  
    public static void main(String args[]) {  
        for(int x = 10; x < 19; x = x+1) {  
            System.out.print("value of x : " + x );  
            System.out.print("\n");  
        }  
    }  
}
```

Output

value of x : 10
value of x : 11
value of x : 12
value of x : 13
value of x : 14
value of x : 15
value of x : 16
value of x : 17
value of x : 18

Cont...

Java for-each Loop (**Enhanced for loop in Java**)

- As of Java 5, the **enhanced for loop** was introduced.
- This is mainly **used for Arrays**.
- The syntax of enhanced for loop is:

```
for(declaration : expression) {  
    //Statements  
}
```

Cont...

Declaration:

- The newly declared block variable, which is of a **type compatible with the elements of the array** you are accessing.
- The variable will be available within the for block and its **value would be the same as the current array element**.

Expression:

- This **evaluates to the array** you need to loop through.
- The expression can be **an array variable** or **method call** that **returns an array**.

Example

```
public class Test {  
    public static void main(String args[]){  
        String [] names ={"James", "Larry", "Tom"};  
        for( String name : names ) {  
            System.out.print(name);  
            System.out.print(",");  
        }  
    }  
}
```

Output

James, Larry, Tom,

3.4 - Breaking control statement

The break Keyword

- The **break** keyword is used to **stop** the **entire loop**.
- The **break** keyword **can** be **used inside any loop** or a **switch statement**.
- The **break** keyword will **stop the execution** of the **innermost loop** and **start executing** the **next line** of code **after the block**.
- The syntax of a break is a single statement inside any loop:

```
break;
```

Example

```
public class Test {  
    public static void main(String args[]) {  
        int [] numbers = {10, 20, 30, 40, 50};  
        for(int x : numbers) {  
            if(x == 30) {  
                break;  
            }  
            System.out.print(x);  
            System.out.print("\n");  
        }  
    }  
}
```

Output

10

20

Cont...

The continue Keyword

- The **continue** keyword can be used in any of the **loop control structures**. It causes the loop to **immediately jump** to the **next iteration** of the loop.
- In a **for** loop, the **continue** keyword causes flow of control to **immediately jump** to the **update statement**.
- In a **while** loop or **do/while** loop, flow of control **immediately jumps** to the **Boolean expression**.
- The syntax of a continue is

```
continue;
```

Example

```
public class Test {  
    public static void main(String args[]) {  
        int [] numbers = {10, 20, 30, 40, 50};  
        for(int x : numbers ) {  
            if(x == 30) {  
                continue;  
            }  
            System.out.print( x );  
            System.out.print("\n");  
        }  
    }  
}
```

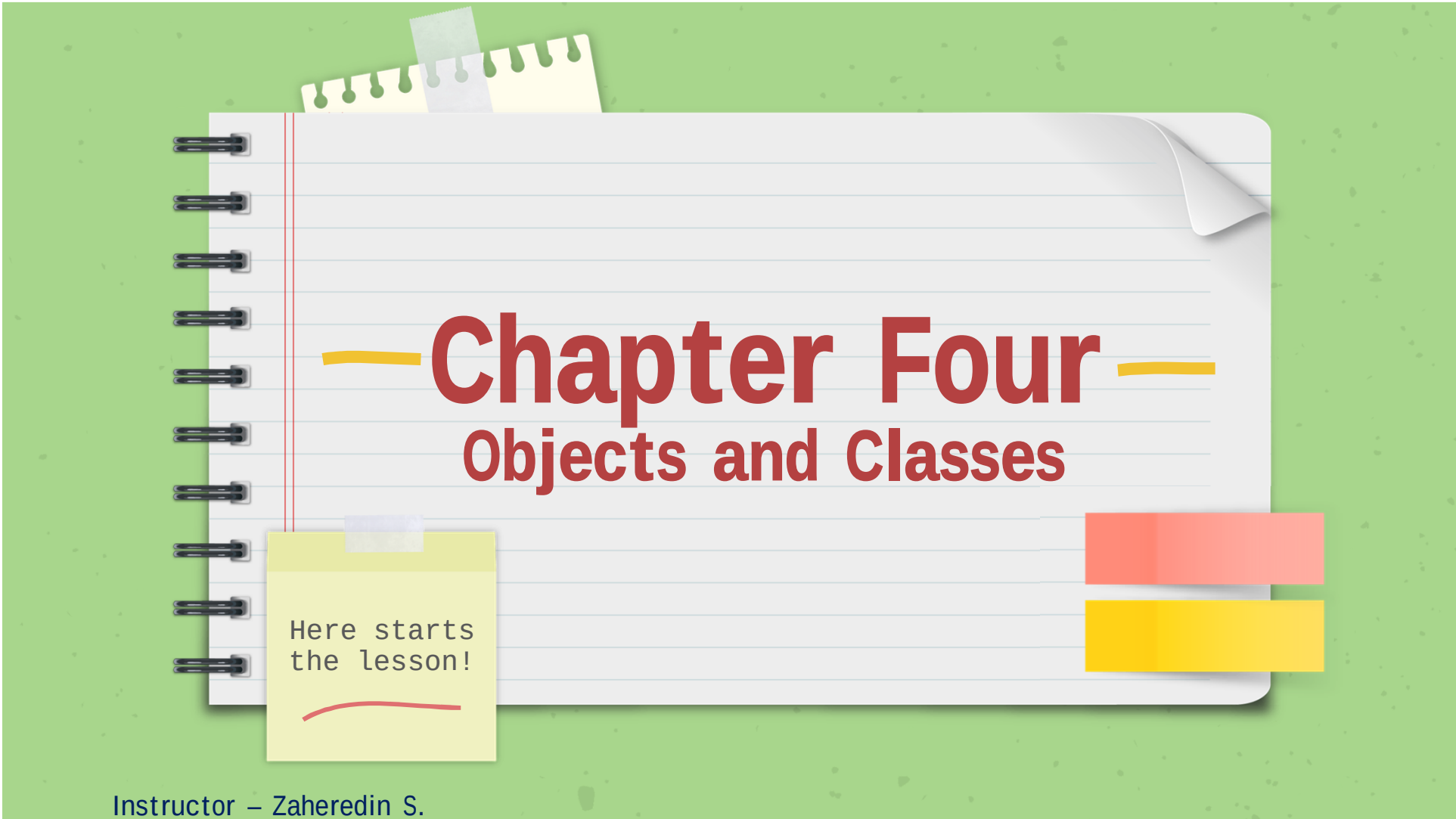
Output

10
20
40
50



END

Thank You!



— Chapter Four —

Objects and Classes

Here starts
the lesson!

Instructor – Zaheredin S.

Chapter Four Contents!



4.1 – Object variables

4.2 – Defining a class

Chapter Four Contents!



4.3 – Instantiating and using objects

4.4 – Instance fields, Constructors and methods

4.1 – Object Variables

- We know that, java is an **Object-Oriented** programming language.
- As a language that has the Object Oriented feature, **Java supports** the following **fundamental concepts**:
 - ❖ Polymorphism
 - ❖ Inheritance
 - ❖ Encapsulation
 - ❖ Abstraction
 - ❖ Classes
 - ❖ Objects
 - ❖ Method
 - ❖ Message Parsing

Cont...

Object in java

- An entity that has **state** and **behavior** is known as an **object**. It can be **physical** or **logical** (tangible and intangible).
- The example of intangible object is **banking system**.

Let us now look deep into what are objects.

- If we consider the real world we can find many objects around us, **Cars**, **Dogs**, **Humans**, etc. All these objects have a **state** and **behavior**.

Cont...

- If we consider a **dog**, then its state is - **name**, **breed**, **color**, and the behavior is - **barking**, **wagging**, **running**.
- Another Example: **Pen** is an object. Its name is **Reynolds**, color is **white** etc. known as its state. It is used to **write**, so **writing** is **its behavior**.
- If you compare the **software object** with a **real world object**, they have very **similar characteristics**.
- Software objects also have a **state** and **behavior**.
- A software object's state is stored in **fields** and behavior is shown via **methods**.

Cont...

- Therefore, in software development, **methods** operate on the **internal state of an object** and the **object-to-object communication** is done via **methods**.
- Object is an **instance of a class**. Class is a **template** or **blueprint** from which objects are **created**. So object is the instance (result) of a class.

4.2 - Defining a class

Classes in Java

- A class is a group of objects that has common properties and behaviour.
- A class is a blue print from which individual objects are created or it describes the behaviors/states that object of its type support.
- Syntax to declare a class:

```
class <class_name>{  
    data member;  
    method;  
}
```

A sample of a class is given below

```
class Dog{  
    String breed;  
    String color;  
    int age;  
    void barking(){  
  
    }  
    void hungry(){  
  
    }  
}
```

A class can contains

- Any number of methods to access the value of various kinds of methods. In the above example, `barking()`, `hungry()` are methods.
- data member
- constructor
- block
- class and interface

4.3 - Instantiating and using objects

Creating an Object

- In java, **creating an object** of the class is called **instantiation**.
- As mentioned previously, a **class** provides the **blueprints for objects**.
- Therefore, an **object** is **created** from a **class**.
- In Java, the **new key word** is used to create new objects.
- Syntax for object instantiation.

```
ClassName objName = new ClassName();
```

Cont...

Example of **creating** and **using an object** is given below:

```
class Student{  
    int id;  
    String name;  
    public static void main(String []args){  
        // creating an object std  
        Student std1 = new Student();  
        System.out.println(std1.id);  
        System.out.println(std1.name);  
    }  
}
```

Output

0

null

4.4 - Instance fields, Constructors and methods

1. Instance variable in Java

- A variable that is created **inside the class** but **outside the method**, is known as instance variable.
- Instance variable **doesn't get memory at compile time**.
- It gets memory at **runtime** when **object** (instance) **is created**. That is why, it is known as instance variable.

2. Method in Java

- In java, a method is like **function** i.e. used to represent **behaviour of an object**.

Cont...

- A method is a **block of code** or **collection of statements** or a set of code **grouped together** to perform a certain task or operation.

Cont...

3. Constructors

- in java, constructor is a **special type of method** that is used to **initialize the object**.
- It is **invoked** at the time of **object creation**.
- It **constructs the values** i.e. provides **data** for the **object** that is why it is known as **constructor**.
- Every **class** has a **constructor**.
- If we **do not explicitly write a constructor** for a class, the **Java compiler** builds a **default constructor** for that class.

Cont...

- Each time a **new object** is **created**, at least **one constructor** will be **invoked**.
- A **class** can have **more than** one constructor.
- A constructor is called **automatically** when a **new instance of an object** is created.

Here's the basic format for coding a constructor:

```
public ClassName (parameter-list) {  
    // statements  
}
```

Cont...

- Constructors have **one purpose in life**: to create an **instance of a class**. This can also be called **creating an object**.
- The **purpose of methods** is much more **general**.
- A method's **basic function** is to **execute Java code**.

Cont...

Example of a constructor is given below:

```
class Puppy{  
    public Puppy(){  
  
    }  
    public Puppy(String name){  
        // This constructor has one parameter, name.  
    }  
}
```

- Java also supports **Singleton Classes** where you would be able to create **only one instance** of a class.

Cont...

There are two types of constructors

- ❖ Default constructor (no-arg constructor)
- ❖ Parameterized constructor

I. Java Default Constructor

- A constructor that have no parameter is known as default constructor.
- Syntax of default constructor:

```
<class_name>(){  
}
```

Cont...

Example of default constructor

```
class Dog1{  
    Dog1(){  
        System.out.println("Dog is created");  
    }  
    public static void main(String args[]){  
        Dog1 d = new Dog1();  
    }  
}
```

Output
Dog is created

Cont...

- **Default constructor** provides the **default values** to the object like **0**, **null** etc. depending on the **type**.

```
class Student{  
    int id;  
    String name;  
    void Display(){  
        System.out.println(id+" "+name);  
    }  
    public static void main(String args[]){  
        Student s=new Student();  
        s.Display();  
    }  
}
```

Output
0 null

Cont...

- **Explanation:** In the above class, you are **not creating any constructor** so compiler provides you a **default constructor**. Here **0** and **null** values are provided by default constructor.

II. Java parameterized constructor

- A constructor that **have parameters** is known as **parameterized constructor**.

Why use parameterized constructor?

- **Parameterized constructor** is used to provide **different values** to the **distinct objects**. Example:

Cont...

```
class Student{
    int id;
    String name;
    public Student(int i, String n){
        id = i;
        name = n;
        System.out.println(id+" "+name);
    }
    public static void main(String []args){
        Student std = new Student(111, "tommy");
    }
}
```

Output

111 tommy

Difference between constructor and method

Constructor

- It is used to initialize the state of an object.
- Must not have return type.
- It is invoked implicitly.
- The java compiler provides a default constructor if you don't have any constructor.
- Constructor name must be same as the class name.

Method

- It is used to expose behaviour of an object.
- Method must have return type.
- Method is invoked explicitly.
- It is not provided by compiler in any case.
- Method name may or may not same as class name.

Cont...

Accessing Instance Variables and Methods

- Instance variables and methods are accessed via created objects.
- To access an instance variable and method the fully qualified path should be as follows:

// First create an object

```
ObjectReference = new Constructor ();
```

// Now call a variable as follows

```
ObjectReference.variableName;
```

// Now you can call a class method as follows

```
ObjectReference.MethodName();
```

Cont...

Example to access instance variables & methods.

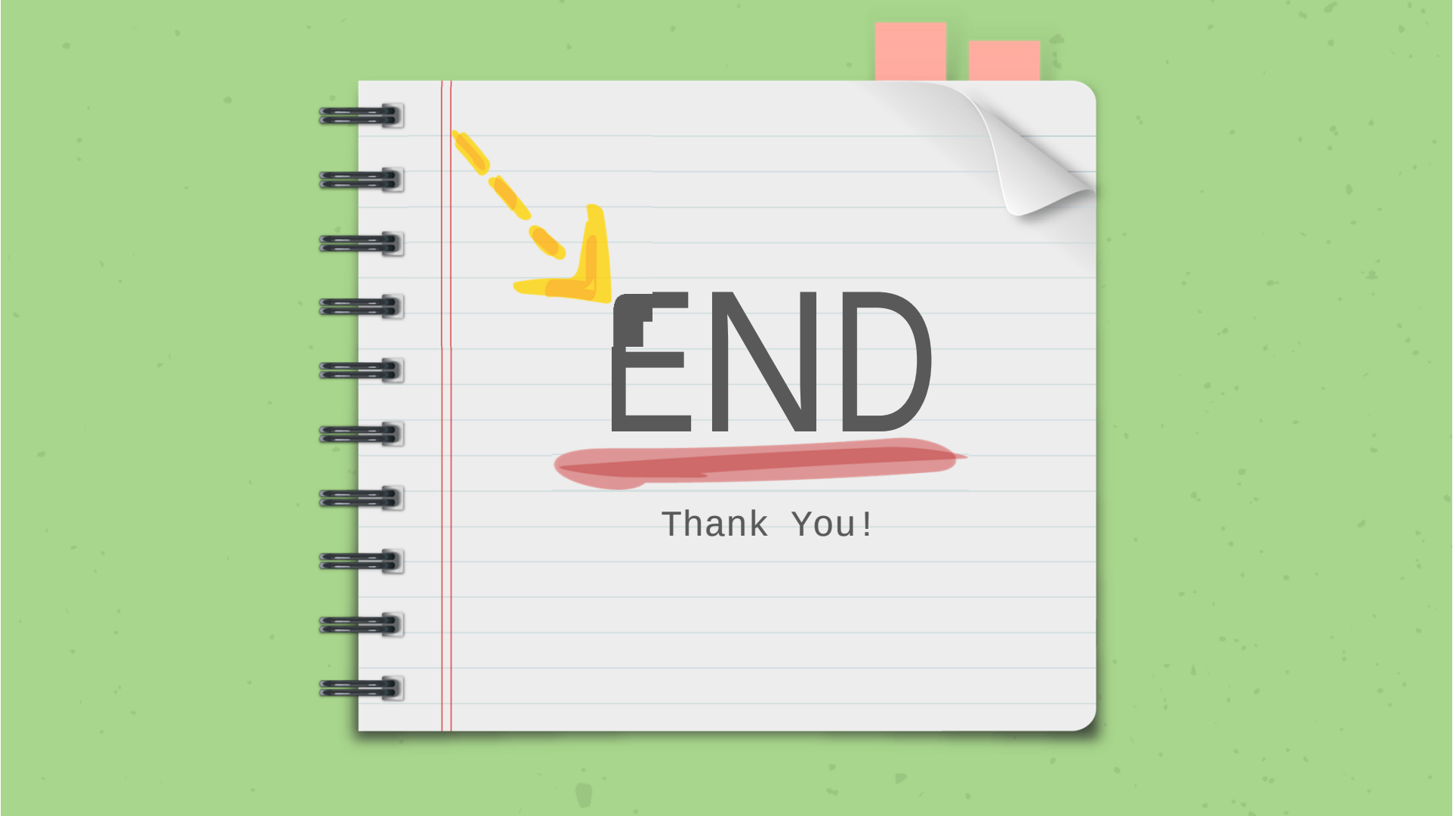
```
class Puppy{  
    int puppyAge;  
    public Puppy(String name){  
        System.out.println("Passed Name is :" + name);  
    }  
    public void setAge(int age){  
        puppyAge = age;  
    }  
    public int getAge(){  
        return puppyAge;  
    }  
}
```

Cont...

```
public static void main(String []args){  
    Puppy p = new Puppy("tommy");  
    //Call class method to set puppy's age  
    p.setAge(2);  
    //Call another class method to get puppy's age  
    p.getAge();  
    //You can access instance variable as follows as well  
    System.out.println(" Puppy's age is :" + p.puppyAge);  
}  
}
```

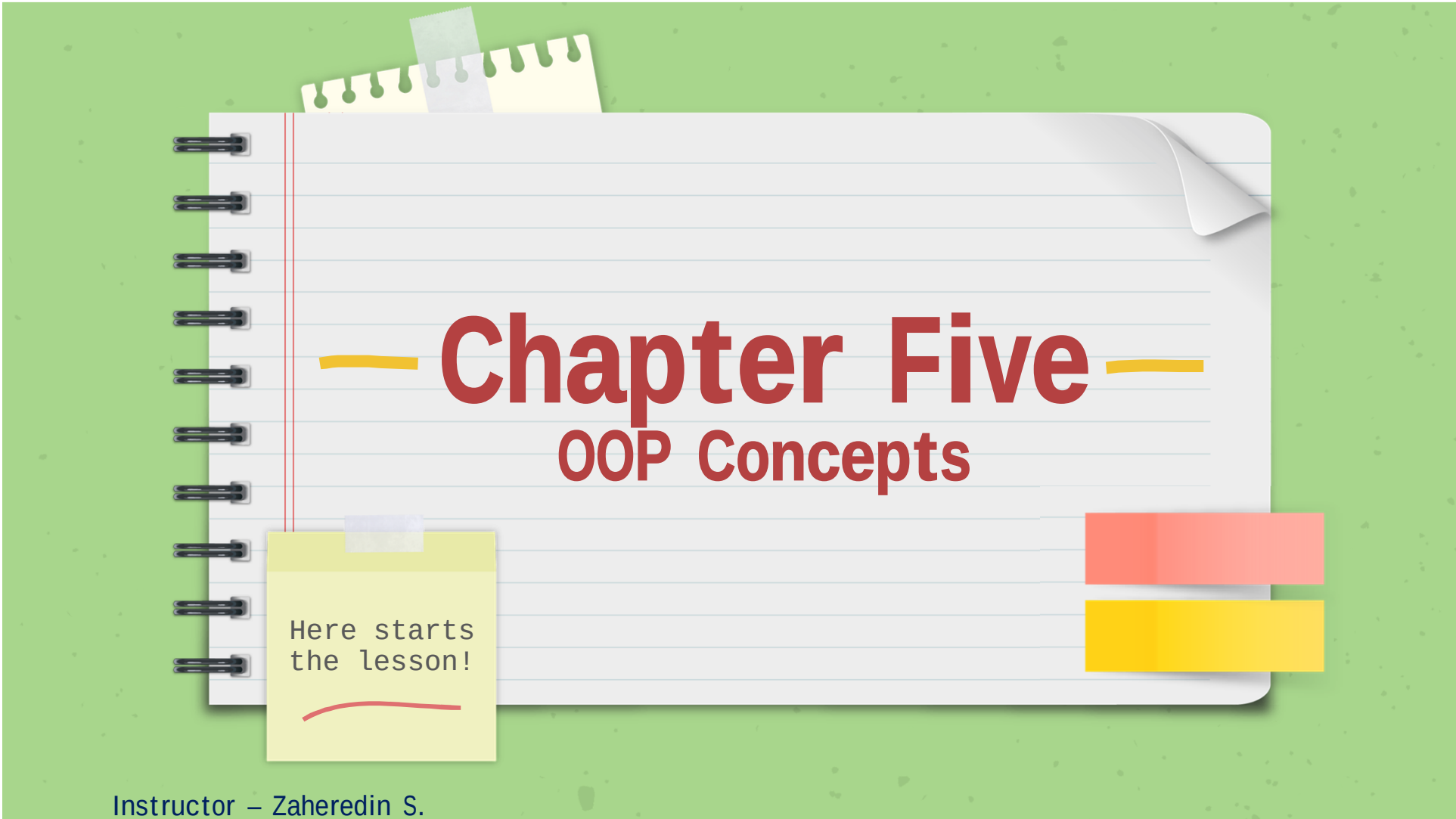
Output

Passed Name is :tommy
Puppy's age is :2



END

Thank You!



Chapter Five

OOP Concepts

Here starts
the lesson!

Instructor – Zaheredin S.

Chapter Five Contents!



5.1 – Encapsulation

5.2 - Inheritance

Chapter Five Contents!



5.3 – Polymorphism

**5.4 – Abstract classes and
Interfaces**

5.1 – Encapsulation

- **Encapsulation** is one of the **four fundamental OOP** concepts.
- In java, it is a **process of wrapping methods** (code) and **variables** (data) **together** inside a single class. Or,
- To make sure that "**sensitive**" data is hidden from users.
- It prevents **outer classes** from **accessing** and **changing data** and **methods** of a class. This also helps to achieve **data hiding**.
- We can create a **fully encapsulated class** in java by making **all the data members** of the class **private**.

Cont...

- Now we can use **setter** and **getter** methods to **set** and **get** the **data** in it.
- It is the **technique** of making the **fields** in a class **private** and providing **access** to the **fields** via **public** **methods**.
- If a **field** is declared **private**, it **cannot be accessed** by anyone **outside** the **class**, thereby **hiding** the **fields** within the class.
- For this reason, **encapsulation** is also referred to as **data hiding**.

Look at example that depicts encapsulation

```
public class EncapTest {  
    private String name;  
    private int age;  
    public int getAge(){  
        return age;  
    }  
    public String getName(){  
        return name;  
    }  
}
```

```
    public void setAge(int  
newAge){  
        age = newAge;  
    }  
    public void setName(String  
newName){  
        name = newName;  
    }  
}
```

Cont...

- The **public methods** are the **access points** to this **class fields** from the **outside java world**.
- Normally, these methods are referred as **getters** and **setters**.
- Therefore, **any class** that **wants to access** the **variables** **should access them** through these **getters** and **setters**.

Cont...

```
/*File name : RunEncap.java*/  
public class RunEncap{  
    public static void main(String args[]){  
        EncapTest encap = new EncapTest();  
        encap.setName("James");  
        encap.setAge(20);  
        encap.setIdNum("12343ms");  
        System.out.print("Name : " + encap.getName()+ " Age  
        :"+encap.getAge());  
    }  
}
```

Output

Name : James Age : 20

5.2 – Inheritance

I. Inheritance

- It is one of the **fundamental concepts** of OOP, that allows us to **create a new class** from an **existing class**.
- The **new created class** is known as **subclass** (child/derived class) and the **existing class** is known as **superclass** (parent/base class).
- Inheritance can be defined as the **process** where **one object acquires** the **properties of another**.

Cont...

- When we talk about inheritance, the most commonly used keyword would be **extends** and **implements**.
- These words would **determine** whether **one object IS-A type** of **another**.

IS-A Relationship:

- In java, **inheritance** is an **is-a relationship**. That is, we use inheritance **only if there exists an is-a relationship** between **two classes**.
- IS-A is a way of saying: **This object is a type of that object**.

Cont...

Let us see how the extends keyword is used to achieve inheritance.

```
public class Animal{  
}  
public class Mammal extends Animal{  
}  
public class Reptile extends Animal{  
}  
public class Dog extends Mammal{  
}
```

Cont...

Now, based on the above example, the following are true:

- Animal is the superclass of Mammal class.
- Animal is the superclass of Reptile class.
- Mammal and Reptile are subclasses of Animal class.
- Dog is the subclass of both Mammal and Animal classes.

Now, if we consider the IS-A relationship, we can say:

- Mammal IS-A Animal
- Reptile IS-A Animal
- Dog IS-A Mammal
- Hence : Dog IS-A Animal as well

Cont...

- With use of the extends keyword the subclasses will be able to inherit all the properties of the super class except for the private properties of the super class.

Cont...

```
class Animal {  
    // field and method of the parent class  
    String name;  
    public void eat() {  
        System.out.println("I can eat");  
    }  
}  
  
// inherit from Animal  
class Dog extends Animal {  
    public void display() { // new method in subclass  
        System.out.println("My name is " + name);  
    }  
}
```

Cont...

```
class Main {  
    public static void main(String[] args) {  
        // create an object of the subclass  
        Dog d = new Dog();  
        // access field of superclass  
        d.name = "Boby";  
        d.display();  
        // call method of superclass using object of subclass  
        d.eat();  
    }  
}
```

Output

My name is Boby
I can eat

Cont...

Now, let us look into how the implements keyword is used to get the IS-A relationship.

- **Classes** to **inherit** from **interfaces** use the **implements** keyword.
- The **classes** can **never extend interfaces**.

```
public interface Animal {  
}  
public class Mammal implements Animal{  
}  
public class Dog extends Mammal{  
}
```

Cont...

- A very important fact to remember is that Java supports only **single inheritance**.
- This means that a **class cannot extend more than one class**.
- Therefore following is illegal:

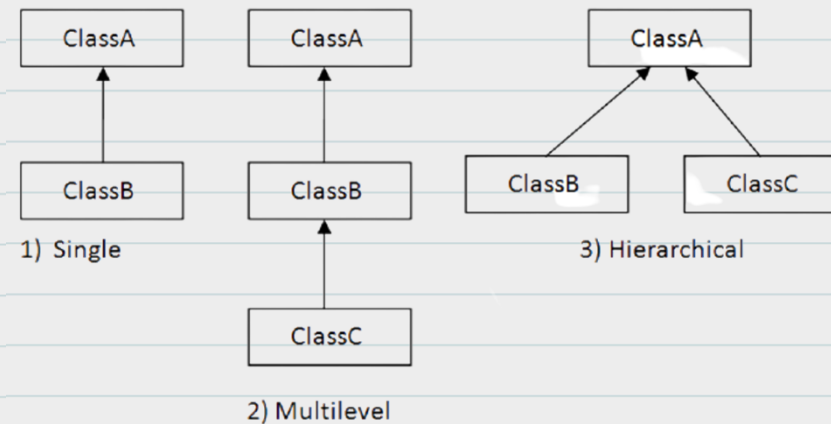
```
public class extends Animal, Mammal{}
```

- However, a **class can implement one or more interfaces**. This has made Java get rid of the **impossibility of multiple inheritances**.

Cont...

Types of Inheritance

- On the basis of class, there can be **three types of inheritance**: **single**, **multilevel** and **hierarchical** in java.
- In java programming, **multiple** and **hybrid inheritance** is supported through **interface only**.



5.3 - Polymorphism

- It is one of the OOPs feature that allows us to perform a single action in different ways.
- It is the capability of a method to do different things based on the object that it is acting upon.
- Polymorphism in Java has two types: Compile time polymorphism (static binding) and Runtime polymorphism (dynamic binding).
- Method overloading is an example of static polymorphism, while method overriding is an example of dynamic polymorphism.

Cont...

Static Polymorphism:

- It is achieved through **method overloading**.
- Method overloading means there are several **methods** present **in a class** having the **same name** but different **types/order/number** of **parameters**.
- At **compile time**, Java knows **which method** to **invoke** by **checking** the **method signatures**.
- So, this is called **compile time polymorphism** or **static binding**.

The concept will be clear from the following example:

Cont...

```
class DemoOverload{  
    public int add(int x, int y){ //method 1  
        return x+y;  
    }  
    public int add(int x, int y, int z){ //method 2  
        return x+y+z;  
    }  
    public int add(double x, int y){ //method 3  
        return (int)x+y;  
    }  
}
```

Cont...

```
class Test{  
    public static void main(String[] args){  
        DemoOverload demo=new DemoOverload();  
        System.out.println(demo.add(2,3));    //method 1 called  
        System.out.println(demo.add(2.5,3)); //method 3 called  
        System.out.println(demo.add(2,3,4)); //method 2 called  
    }  
}
```

Cont...

Dynamic Polymorphism:

- Suppose a **sub class overrides** a particular **method** of the **super class**.
- Let us say, in the program we create an **object of the subclass** and assign it to the **super class reference**.
- Now, if we call the **overridden method** on the **super class reference** then the **subclass** version of the **method** will be **called**.

Example

Cont...

```
class Vehicle{  
    public void move(){  
        System.out.println("Vehicles can move!!");  
    }  
}  
class MotorBike extends Vehicle{  
    public void move(){  
        System.out.println("MotorBike can move & accelerate too");  
    }  
}
```

Cont...

```
class Test{  
    public static void main(String[] args){  
        Vehicle vh=new MotorBike();  
        vh.move();  
        vh=new Vehicle();  
        vh.move();  
    }  
}
```

Output

MotorBike move & accelerate too
Vehicles can move!!

- It should be noted that in the first call to `move()`, the reference type is `Vehicle` and the object being referenced is `MotorBike`.

Cont...

- So, when a call to `move()` is made, Java waits until **runtime** to determine which object is actually being **pointed to by the reference**.
- In this case, the **object** is of the class **MotorBike**. So, the `move()` method of MotorBike class will be **called**.
- In the second call to `move()`, the object is of the class **Vehicle**. So, the `move()` method of Vehicle will be **called**.
- As the method to call is **determined at runtime**, this is called **dynamic binding** or **late binding**.

5.4 - Abstract classes and Interfaces

I. Abstraction

- The major uses of abstract classes and methods is to achieve abstraction in java.
- Abstraction is an important concepts in OOP that allows us to hide unnecessary details and only show the needed information.
- This allow us to manage complexity by omitting or hiding details with a simpler, higher-level idea.
- Practical example of abstraction can be motorbike brakes. We know what brake does. When we apply the brake, the motorbike will stop. However,

Cont...

- The **working of the brake** is kept hidden from us.
- An **abstract class** is one that **cannot be instantiated**.
- All other **functionality** of the class **still exists**, and its **fields, methods, and constructors** are all **accessed in the same manner**.
- You just **cannot create an instance** of the abstract class.
- If a **class** is **abstract** and **cannot be instantiated**, the class **does not have much use** unless it is subclass.

Cont...

- A parent class contains the **common functionality** of a collection of **child classes**, but the **parent class itself** is **too abstract** to be used on **its own**.
- Use the **abstract** keyword to declare a class abstract.
- The **keyword** appears in the class declaration somewhere **before** the **class keyword**.
- An **abstract class** can **have** both **regular methods** and **abstract methods**.

Cont...

```
public abstract class Animal{  
    public data;  
    public abstract void Run();  
}  
class Dog extends Animal{  
    public data;  
    public void Run(){  
        //method implementation  
    }  
}
```

Cont...

Abstract Methods

- It is a **method** that **doesn't have** its **body**.
- **If** you want a **class** to **contain** a **particular method** but you want the **actual implementation** of that method to be determined by **child classes**, you can declare the **method** in the **parent class** as **abstract**.
- The **abstract** keyword is also used to **declare** a **method** as **abstract**.
- An **abstract method** consists of a **method signature**, but no **method body**.

Cont...

- Abstract method would have no definition, and its signature is followed by a semicolon, not curly braces as follows:

```
public abstract class Employee{  
    private String name;  
    private String address;  
    private int number;  
    //Remainder of class definition  
    public abstract double computePay();  
}
```

Cont...

Declaring a method as abstract has two results:

- ❖ The **class must** also be **declared abstract**. If a class contains an **abstract method**, the **class must** be **abstract** as well.
- ❖ Any **child class** must either **override the abstract method** or declare itself **abstract**.
- A **child class** that **inherits an abstract method** must **override it**. If they **do not**, they must be **abstract** and any of **their children** must **override it**.

Cont...

- Eventually, a descendant class has to implement the abstract method; otherwise, you would have a hierarchy of abstract classes that cannot be instantiated.
- If Dog is extending Animal class, then it is required to implement makeSound() method as follows:

```
public abstract class Animal{  
    public abstract void makeSound();  
    public void eat(){  
        System.out.println("I can eat.");  
    }  
}
```

Cont...

```
public class Dog extends Animal{  
    public void makeSound(){  
        System.out.println("Bark bark");  
    }  
}  
class Main{  
    public static void main(String[] args){  
        Dog d = new Dog();  
        d.makeSound();  
        d.eat();  
    }  
}
```

Output
Bark bark
I can eat.

Cont...

II. Interfaces

- An **interface** is a **fully abstract class**. It includes a collection of **abstract methods**.
- A **class implements** an **interface**, thereby **inheriting the abstract methods** of the **interface**.
- An **interface** is **not** a **class**.
- Writing an **interface** is similar to writing a **class**, but they are **two different** concepts.
- A class describes the **attributes** and **behaviors** of an **object**.

Cont...

- An **interface** contains **behaviors** that a **class** **implements**.
- Unless the **class** that **implements** the **interface** is **abstract**, all the **methods of the interface** need to be defined in the class.

Cont...

Declaring Interfaces:

- The **interface** keyword is used to declare an interface.

Here is a simple example to declare an interface:

```
/* File name : NameOfInterface.java */  
import java.lang.*;    //Any number of import statements  
public interface NameOfInterface{  
    //Any number of final, static fields  
    //Any number of abstract method declarations  
}
```

Cont...

Interfaces have the following properties:

- An **interface** is **implicitly abstract**. You **do not need** to **use** the **abstract** keyword when **declaring** an **interface**.
- **Each method** in an **interface** is also **implicitly abstract**, so the **abstract** keyword is **not needed**.

Cont...

- Methods in an interface are implicitly public.

```
/* File name : Animal.java */  
interface Animal {  
    public void eat();  
    public void travel();  
}
```

Cont...

Implementing Interfaces:

- Like abstract classes, we can not create objects of interface.
- A **class** uses the **implements** keyword to **implement** an **interface**.
- The **implements** keyword appears in the **class declaration** following the **extends** portion of the declaration.
- If a **class** does **not perform all** the **behaviors of the interface**, the **class** must **declare itself as abstract**.

Cont...

```
/* File name : MammalInt.java */  
public class MammalInt implements Animal{  
    public void eat(){  
        System.out.println("Mammal eats");  
    }  
    public void travel(){  
        System.out.println("Mammal travels");  
    }  
    public int noOfLegs(){  
        return 0;  
    }  
}
```

Cont...

```
public static void main(String args[]){  
    MammalInt m = new MammalInt();  
    m.eat();  
    m.travel();  
}  
}
```

Output

Mammal eats
Mammal travels

Cont...

When implementing interfaces there are several rules:

- A **class** can **extend only one class**, but **implement many interfaces** at a time.

Extending Interfaces:

- An **interface** can **extend another interface**, similarly to the way that a class can extend another class.
- The **extends** keyword is used to **extend an interface**, and the **child interface inherits the methods of the parent interface**.
- The following **Sports interface** is extended by **Hockey** and **Football interfaces**.

Cont...

//Filename: Sports.java

```
public interface Sports{  
    public void setHomeTeam(String name);  
    public void setVisitingTeam(String name);  
}
```

//Filename: Football.java

```
public interface Football extends Sports{  
    public void homeTeamScored(int points);  
    public void visitingTeamScored(int points);  
    public void endOfQuarter(int quarter);  
}
```

Cont...

//Filename: Hockey.java

```
public interface Hockey extends Sports{  
    public void homeGoalScored();  
    public void visitingGoalScored();  
    public void endOfPeriod(int period);  
    public void overtimePeriod(int ot);  
}
```

- The **Hockey** interface has **four methods**, but it **inherits two** from **Sports**; thus, a **class** that **implements Hockey** needs to **implement all six methods**.

Cont...

- Similarly, a **class** that **implements** **Football** needs to define the **three methods** from **Football** and the **two methods** from **Sports**.

Extending Multiple Interfaces:

- A Java **class** can only **extend** one **parent class**.
- **Multiple inheritance** is **not allowed** on class basis.
- **Interfaces** are **not classes**, however, an **interface** can **extend** more than one **parent interface**.
- The **extends** keyword is used **once**, and the **parent interfaces** are declared in a **comma-separated** list.

Cont...

- For example, if the **Hockey interface** extended both **Sports** and **Event**, it would be declared as:

```
public interface Hockey extends Sports, Event
```



END

Thank You!